



<https://haoailab.com/cse291-s26/>

CSE/DSC 291: Deep Learning Systems Spring 2026

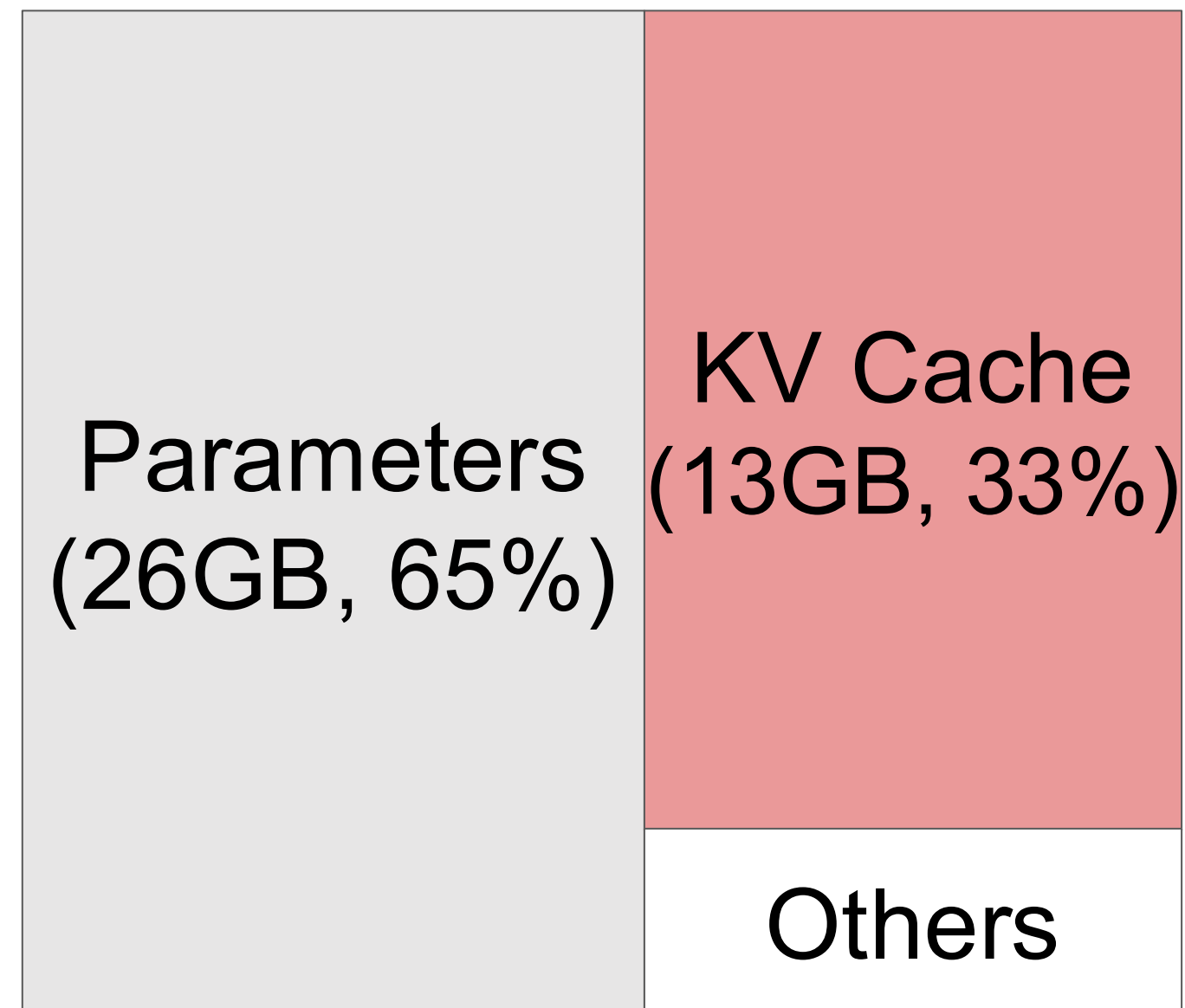
LLM, diffusion, and case studies

Optimizations and Parallelization

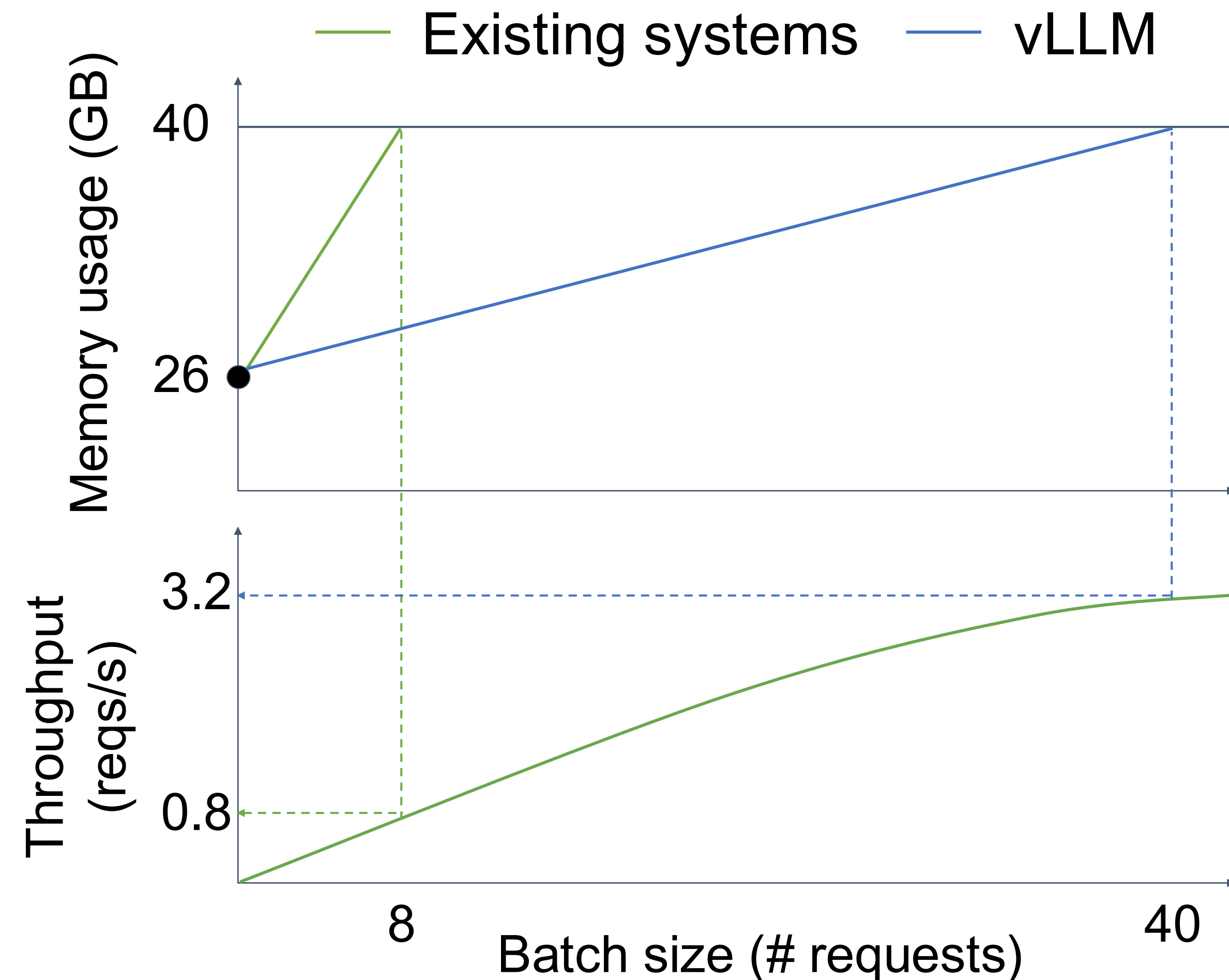
Basics

Key insight

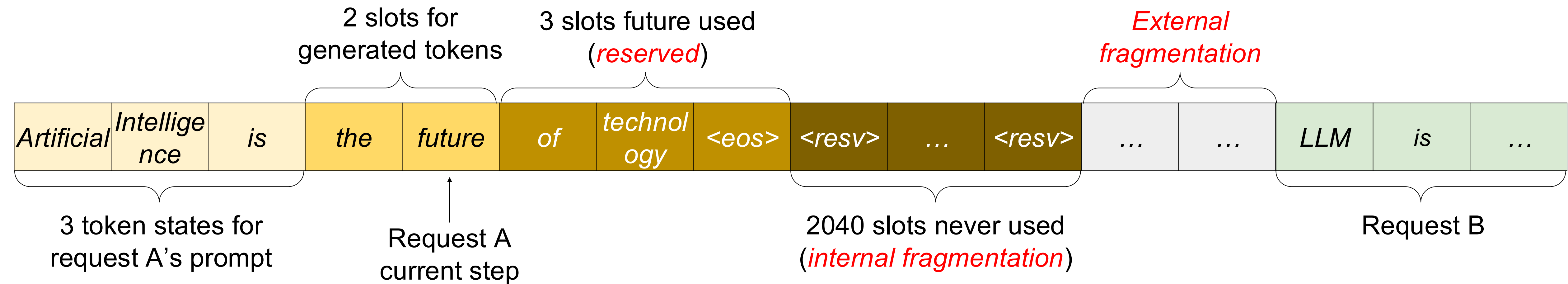
Efficient management of KV cache is crucial for high-throughput LLM serving



13B LLM on A100-40GB

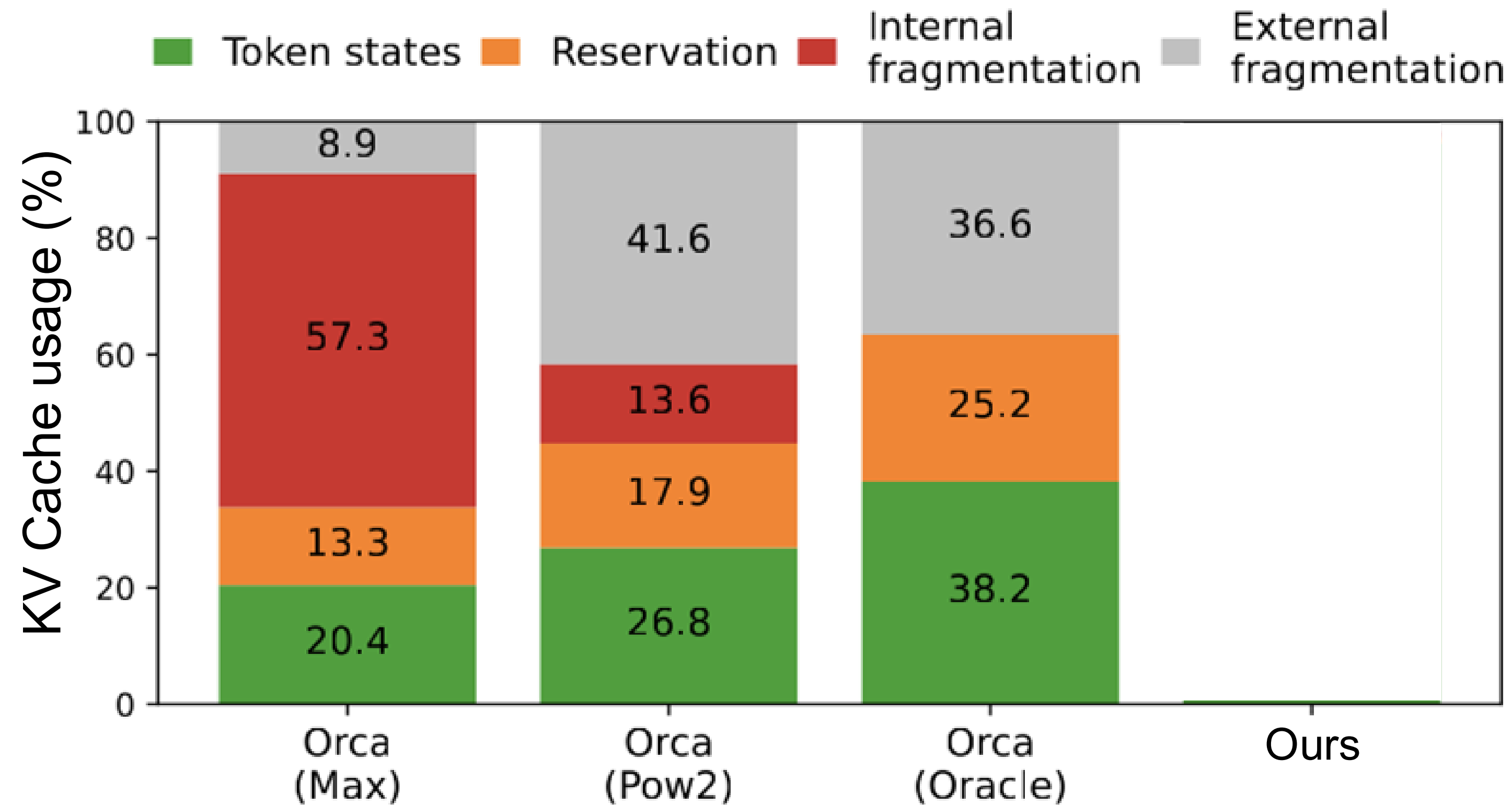


Memory waste in KV Cache



- **Reservation:** not used at the current step, but used in the future
- **Internal fragmentation:** over-allocated due to the unknown output length.

Memory waste in KV Cache

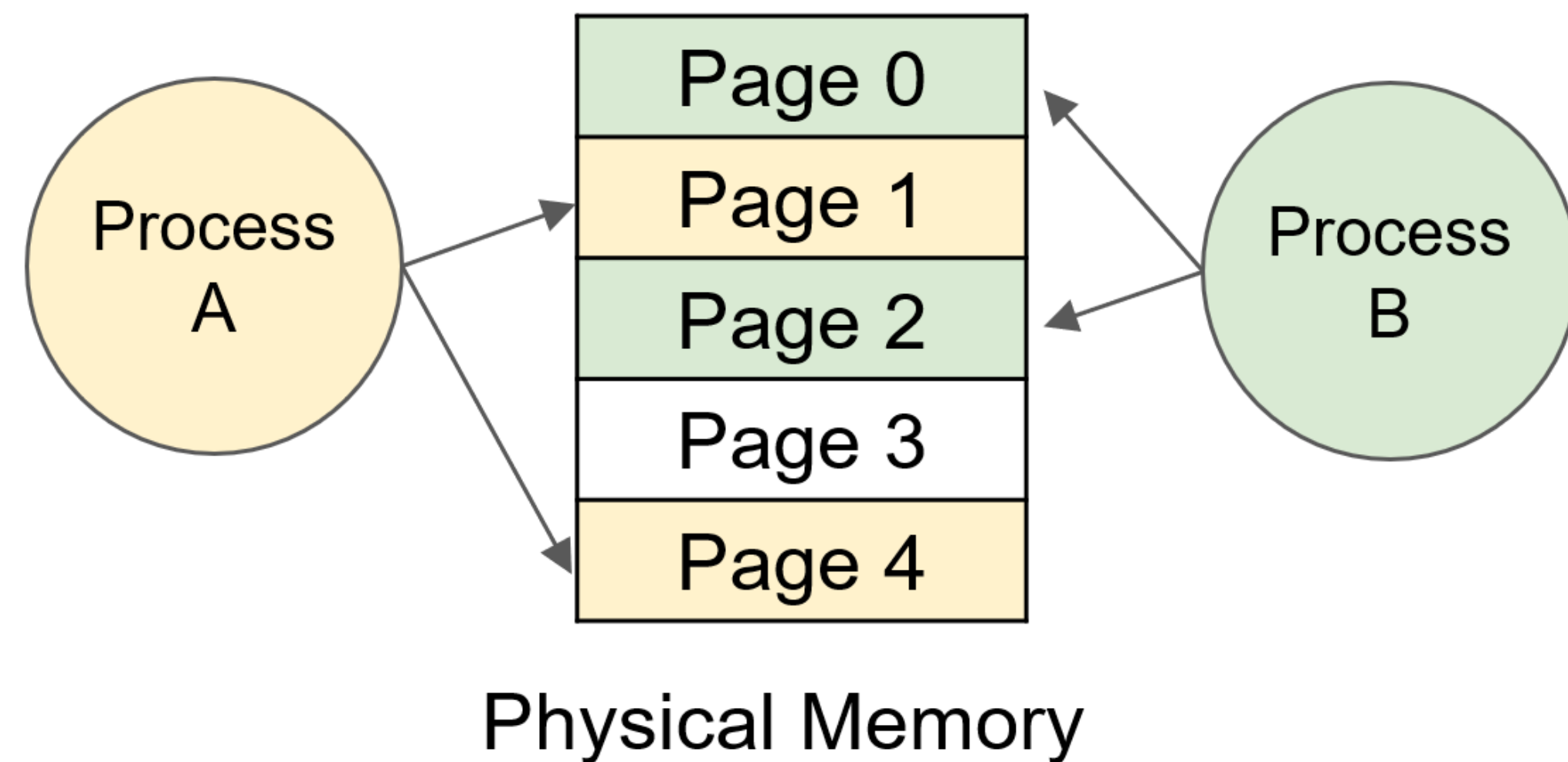


Only **20–40%** of KV cache is utilized to store token states

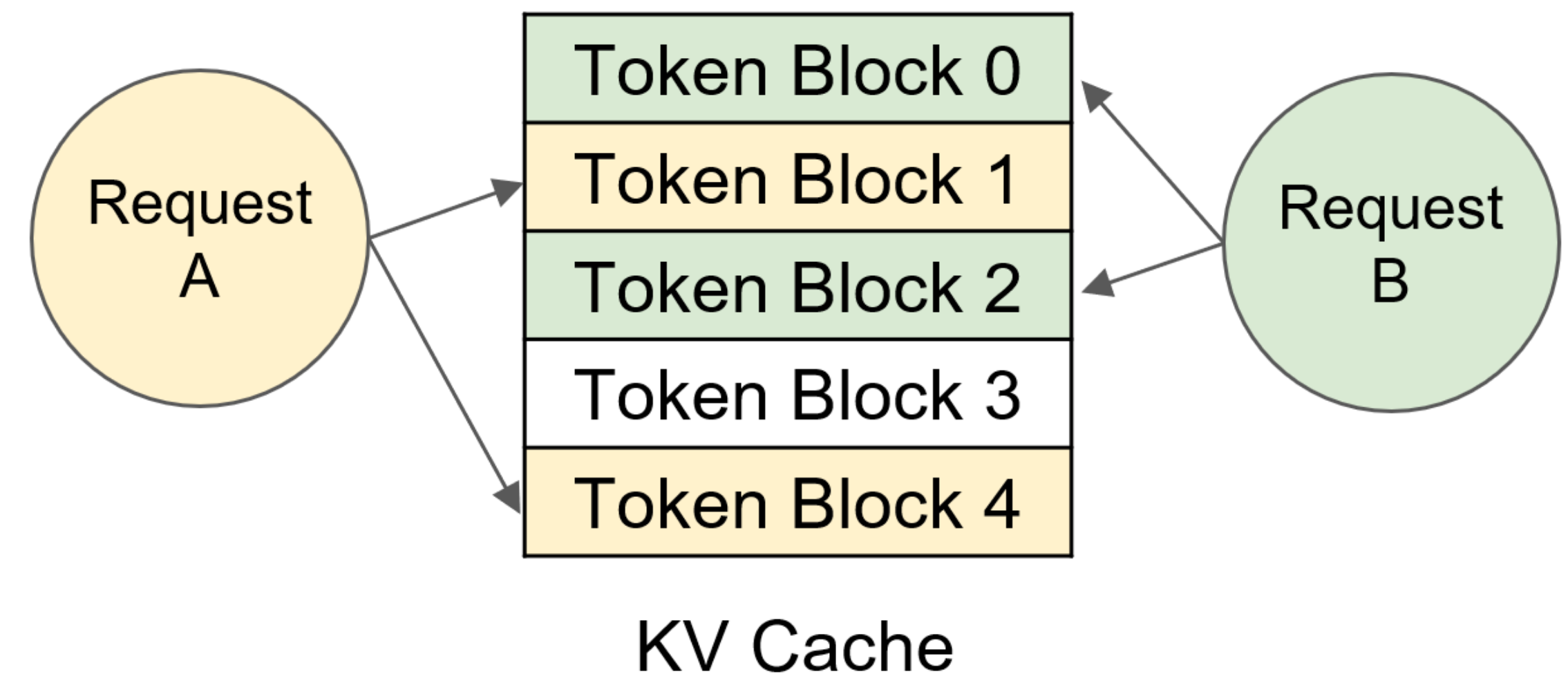
vLLM: Efficient memory management for LLM inference

Inspired by **virtual memory** and **paging**

Memory management in OS

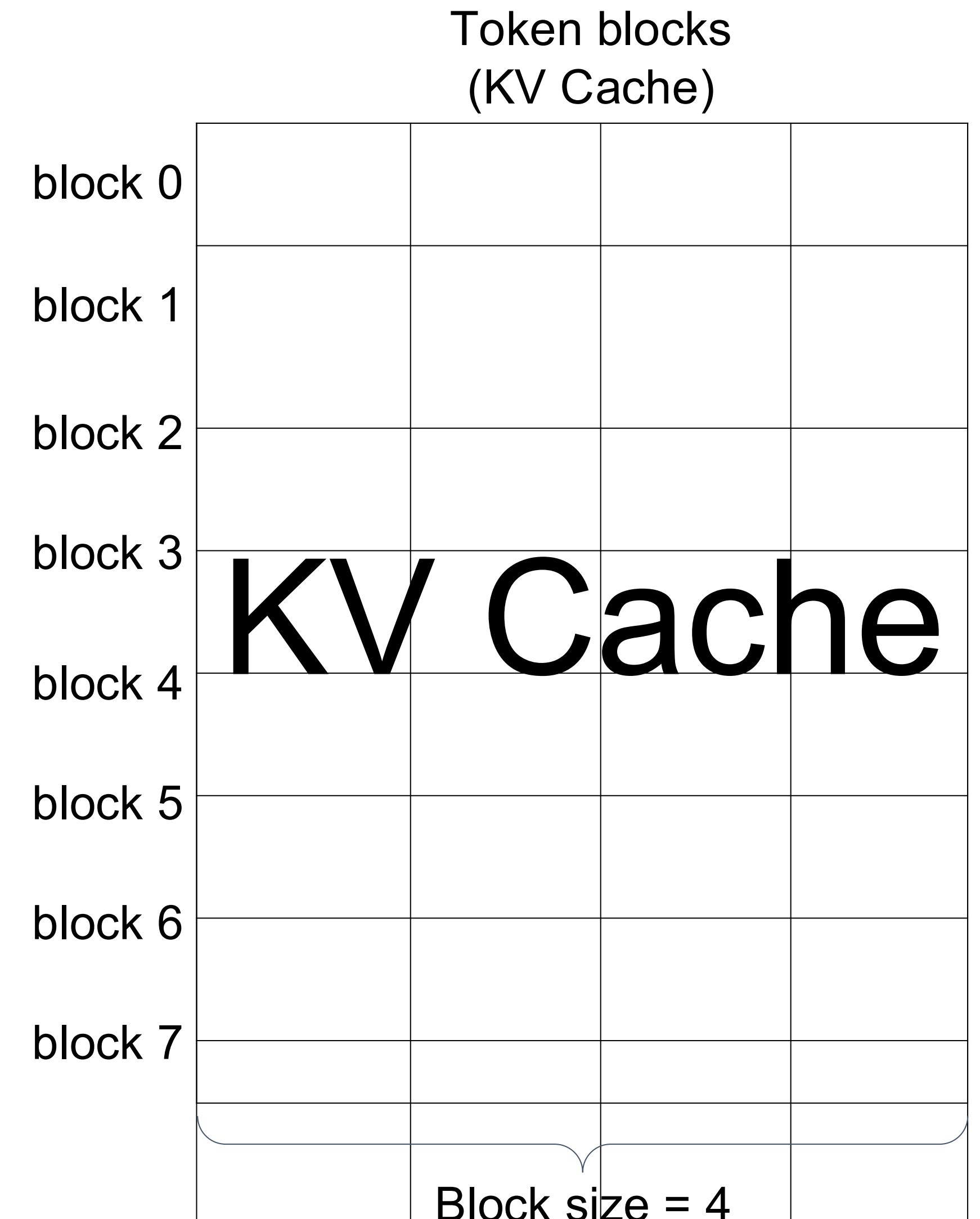


Memory management in vLLM



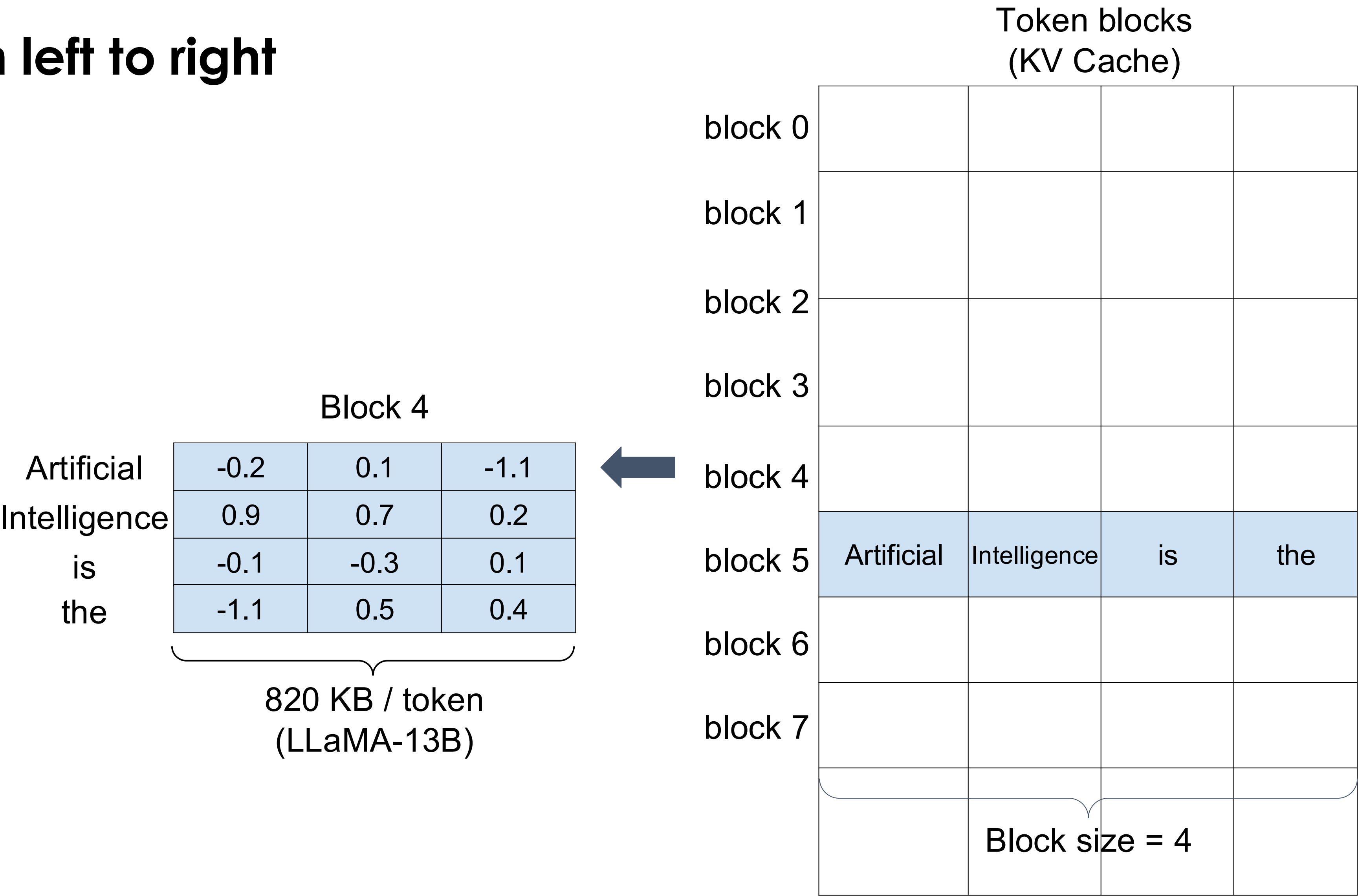
Token block

- A **fixed-size** contiguous chunk of memory that can store token states **from left to right**



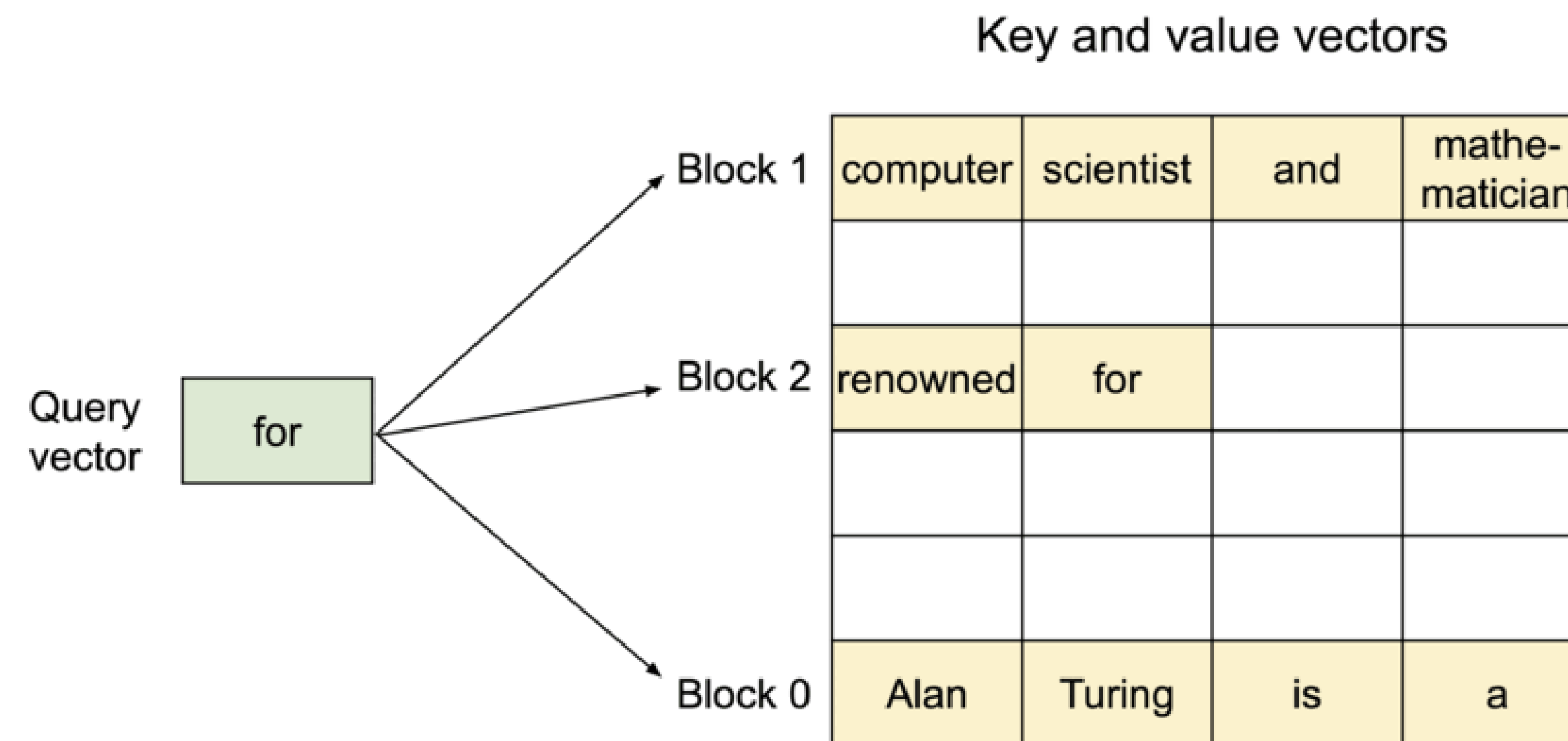
Token block

- A **fixed-size** contiguous chunk of memory that can store token states **from left to right**

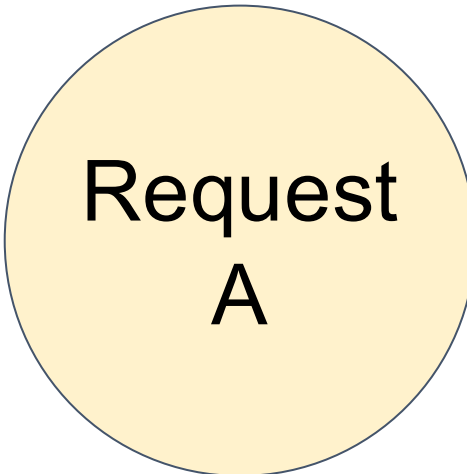


Paged Attention

- An attention algorithm that allows for storing continuous keys and values in non-contiguous memory space



Logical & physical token blocks



Prompt: “Alan Turing is a computer scientist”

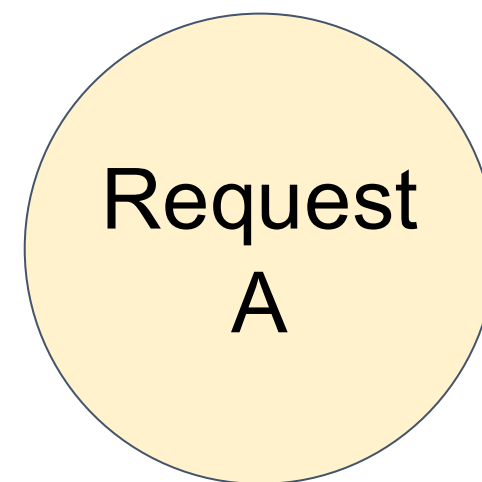
Logical token blocks

block 0	Alan	Turing	is	a
block 1	computer	scientist		
block 2				
block 3				

Physical token blocks
(KV Cache)

block 0				
block 1				
block 2				
block 3				
block 4				
block 5				
block 6				
block 7				

Logical & physical token blocks



Prompt: "Alan Turing is a computer scientist"

Logical token blocks

block 0	Alan	Turing	is	a
block 1	computer	scientist		
block 2				
block 3				

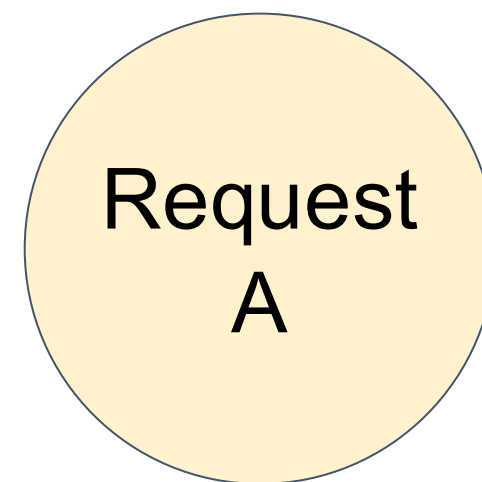
Block table

Physical block number	# Filled
7	4
1	2
—	—
—	—

Physical token blocks
(KV Cache)

block 0				
block 1	computer	scientist		
block 2				
block 3				
block 4				
block 5				
block 6				
block 7	Alan	Turing	is	a

Logical & physical token blocks



Prompt: "Alan Turing is a computer scientist"
Completion: "and"

Logical token blocks

block 0	Alan	Turing	is	a
block 1	computer	scientist		
block 2				
block 3				

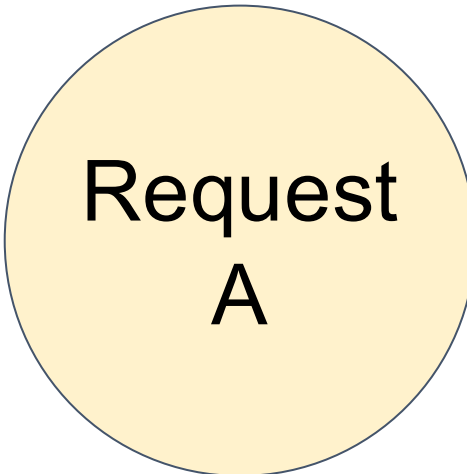
Block table

Physical block number	# Filled
7	4
1	2
—	—
—	—

Physical token blocks
(KV Cache)

block 0				
block 1	computer	scientist		
block 2				
block 3				
block 4				
block 5				
block 6				
block 7	Alan	Turing	is	a

Logical & physical token blocks



Prompt: “Alan Turing is a computer scientist”
Completion: “and”

Logical token blocks

block 0	Alan	Turing	is	a
block 1	computer	scientist	and	
block 2				
block 3				

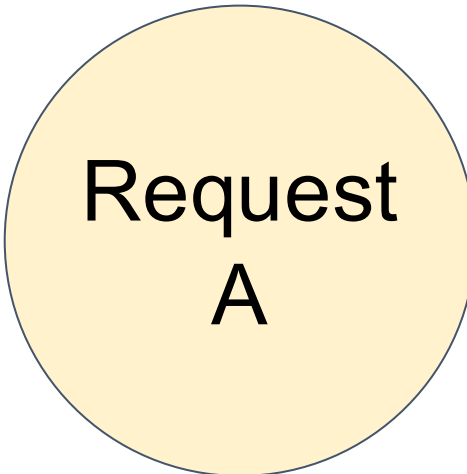
Block table

Physical block number	# Filled
7	4
1	2
—	—
—	—

Physical token blocks
(KV Cache)

block 0				
block 1	computer	scientist		
block 2				
block 3				
block 4				
block 5				
block 6				
block 7	Alan	Turing	is	a

Logical & physical token blocks



Prompt: "Alan Turing is a computer scientist"
Completion: "and"

Logical token blocks

block 0	Alan	Turing	is	a
block 1	computer	scientist	and	
block 2				
block 3				

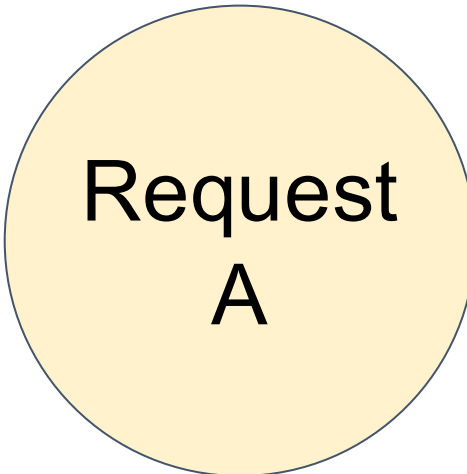
Block table

Physical block number	# Filled
7	4
1	3
—	—
—	—

Physical token blocks
(KV Cache)

block 0				
block 1	computer	scientist	and	
block 2				
block 3				
block 4				
block 5				
block 6				
block 7	Alan	Turing	is	a

Logical & physical token blocks



Prompt: “Alan Turing is a computer scientist”
Completion: “and mathematician”

Logical token blocks

block 0	Alan	Turing	is	a
block 1	computer	scientist	and	mathematician
block 2				
block 3				

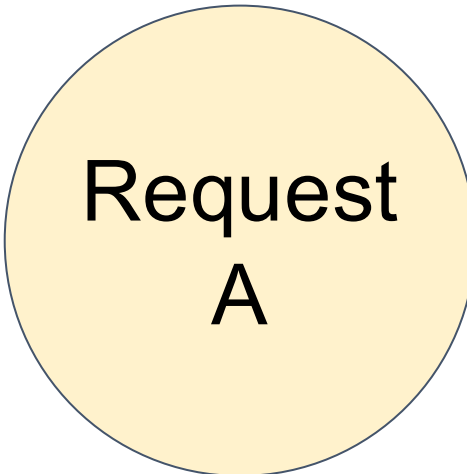
Block table

Physical block number	# Filled
7	4
1	4
—	—
—	—

Physical token blocks (KV Cache)

block 0				
block 1	computer	scientist	and	mathematician
block 2				
block 3				
block 4				
block 5				
block 6				
block 7	Alan	Turing	is	a

Logical & physical token blocks



Prompt: “Alan Turing is a computer scientist”
Completion: “and mathematician renowned”

Logical token blocks

block 0	Alan	Turing	is	a
block 1	computer	scientist	and	mathema tician
block 2	renowned			
block 3				

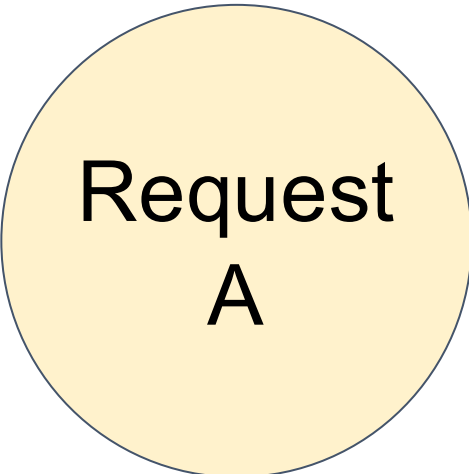
Block table

Physical block number	# Filled
7	4
1	4
5	1
—	—

Physical token blocks
(KV Cache)

block 0				
block 1	computer	scientist	and	mathem atician
block 2				
block 3				
block 4	Allocated on demand			
block 5	renowned			
block 6				
block 7	Alan	Turing	is	a

Serving multiple requests



Block Table

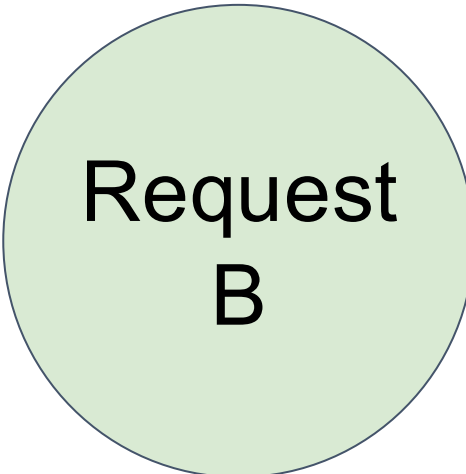
Logical token blocks

Alan	Turing	is	a
computer	scientist	and	mathem atician
renowned			

Physical token blocks
(KV Cache)

computer	scientist	and	mathem atician
Artificial	Intellige nce	is	the
renowned			
future	of	technolog y	
Alan	Turing	is	a

Block Table



Logical token blocks

Artificial	Intelligence	is	the
future	of	technology	

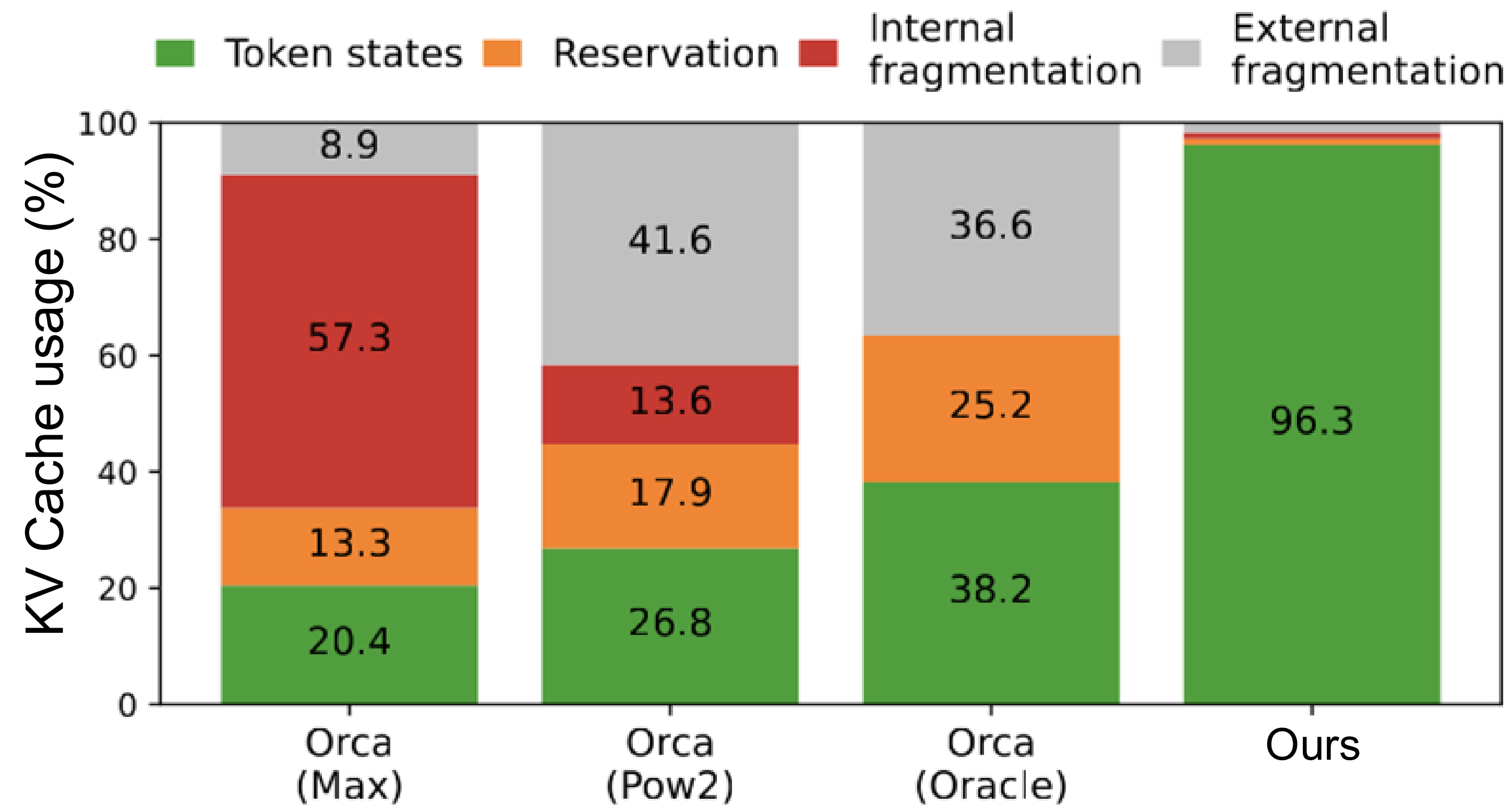
Memory efficiency of vLLM

- Minimal internal fragmentation
 - Only happens at the last block of a sequence
 - **# wasted tokens / seq < block size**
 - Sequence: $O(100)$ – $O(1000)$ tokens
 - Block size: 16 or 32 tokens
- No external fragmentation

Alan	Turing	is	a
computer	scientist	and	mathemati cian
renowned			

Internal fragmentation

Effectiveness of PagedAttention



96.3% KV cache utilization

large b



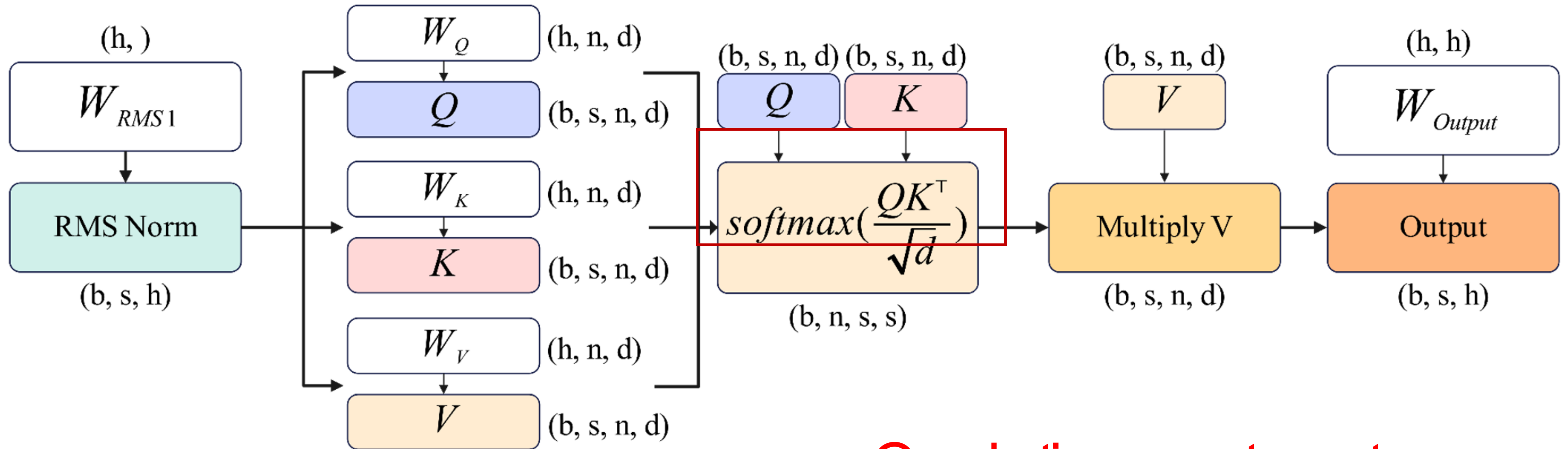
Large Language Models

- LLM Internals
- **Scaling Law**
- Serving and inference optimization
 - Speculative decoding (question in your PA3)
 - **Continuous batching and Paged attention**
 - Disaggregation (in reading)
 - Chunked prefill
 - Kernels.. (guest lecture)

LLM Inference Now is High-stake research topic

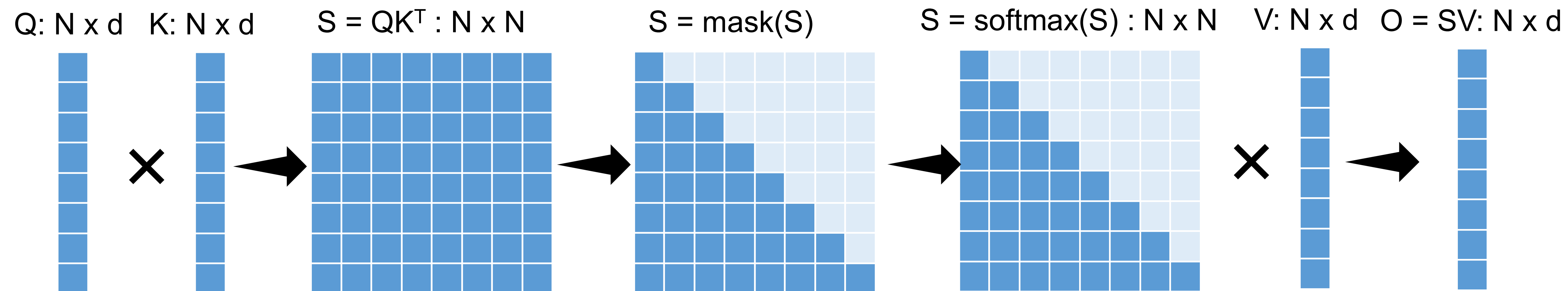
- Scheduling
 - Continuous batching
 - Chunked prefill
 - Disaggregated prefill and decoding
- Speculative Decoding
- Address the memory bottleneck of KV Cache
 - New attention mechanisms: paged, sparse, etc.
 - Sparse KV cache
- Kernel optimizations

The Rest of bottleneck



- Quadratic compute w.r.t. s
- Quadratic memory w.r.t. s

$$\text{Attention: } O = \text{Softmax}(QK^T) V$$



Which would hit bottleneck first

Compute: $4bs^2h$

Memory: bs^2n

Assume $b = 1K, n = 32, h = 4k$

- Assume $s = 4K$ (sequence length)
- compute: $4 * 1K * 4K * 4K * 4K$
 - = 256 Tflops
- memory: $1K * 4K * 4K * 32$
 - = 512G bytes

GPU memory is more scarce than compute at this moment

H100 SXM	
FP64	34 teraFLOPS
FP64 Tensor Core	67 teraFLOPS
FP32	67 teraFLOPS
TF32 Tensor Core*	989 teraFLOPS
BFLOAT16 Tensor Core*	1,979 teraFLOPS
FP16 Tensor Core*	1,979 teraFLOPS
FP8 Tensor Core*	3,958 teraFLOPS
INT8 Tensor Core*	3,958 TOPS
GPU Memory	80GB
GPU Memory Bandwidth	3.35TB/s
Decoders	7 NVDEC 7 JPEG

The Large [bnss] matrix makes thing even worse

Algorithm 0 Standard Attention Implementation

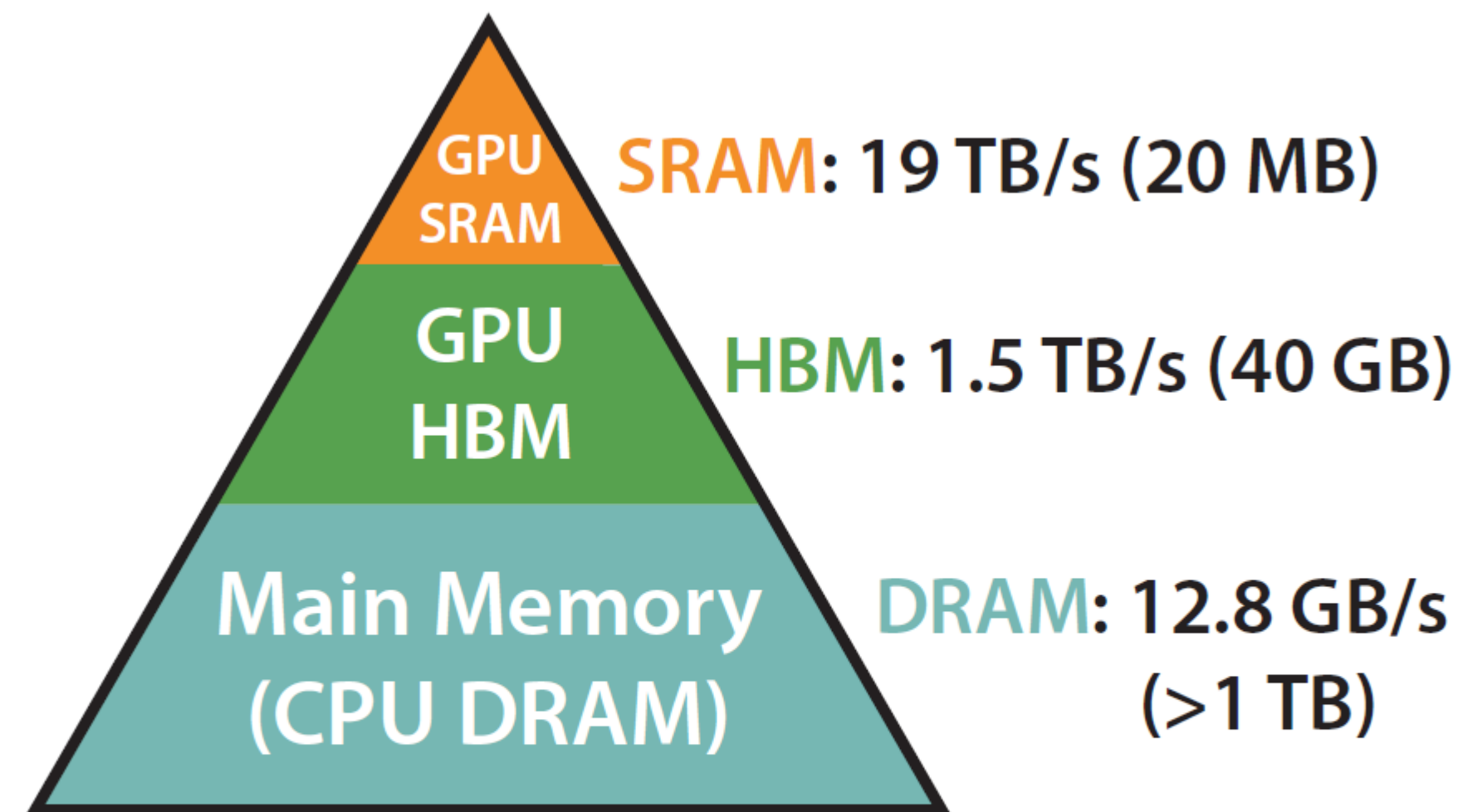
Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM.

- 1: Load $\mathbf{Q}, \mathbf{K}$ by blocks from HBM, compute $\mathbf{S} = \mathbf{QK}^\top$, write $\mathbf{S}$ to HBM.
 - 2: Read $\mathbf{S}$ from HBM, compute $\mathbf{P} = \text{softmax}(\mathbf{S})$, write $\mathbf{P}$ to HBM.
 - 3: Load $\mathbf{P}$ and $\mathbf{V}$ by blocks from HBM, compute $\mathbf{O} = \mathbf{PV}$, write $\mathbf{O}$ to HBM.
 - 4: Return $\mathbf{O}$.
-

Additional Challenges:

- Repeated reads/writes from HBM -> SRAM of the large bnss matrix

Revisit: GPU Memory Hierarchy

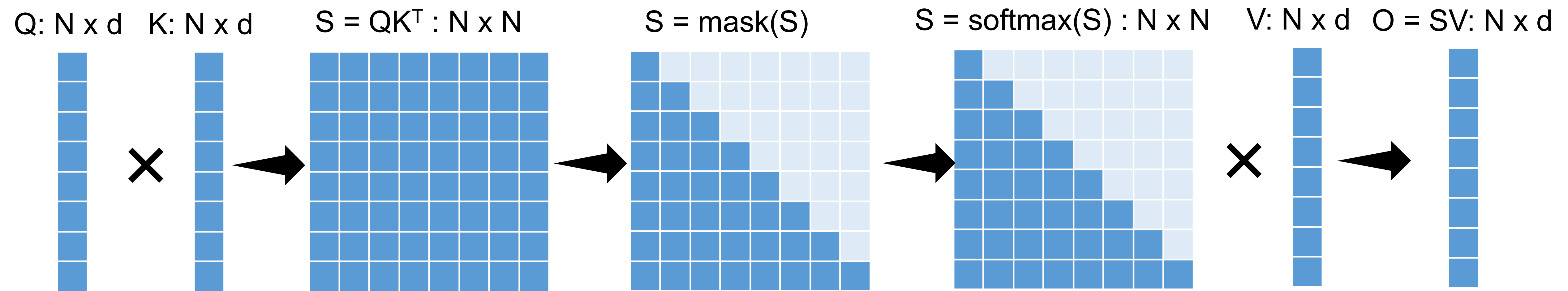


Memory Hierarchy with
Bandwidth & Memory Size



now is 4

Problem: How to tile softmax?



Challenges: We must avoid materializing S while

- Compute softmax reduction O w/o access to $N \times N$ at forward
- Compute backward even without saving the $N \times N$ softmax forward activations

Recall in the Matmul Class

```
dram float A[n/v1][n/v3][v1][v3];
dram float B[n/v2][n/v3][v2][v3];
dram float C[n/v1][n/v2][v1][v2];

for (int i = 0; i < n/v1; ++i) {
    for (int j = 0; j < n/v2; ++j) {
        register float c[v1][v2] = 0;
        for (int k = 0; k < n/v3; ++k) {
            register float a[v1][v3] = A[i][k];
            register float b[v2][v3] = B[j][k];
            c += dot(a, b.T);
        }
        C[i][j] = c;
    }
}
```

- We were able to compute the final results without fully materializing the input / output

The Large [bnss] matrix makes thing even worse

Algorithm 0 Standard Attention Implementation

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM.

- 1: Load $\mathbf{Q}, \mathbf{K}$ by blocks from HBM, compute $\mathbf{S} = \mathbf{Q}\mathbf{K}^\top$, write $\mathbf{S}$ to HBM.
 - 2: Read $\mathbf{S}$ from HBM, compute $\mathbf{P} = \text{softmax}(\mathbf{S})$, write $\mathbf{P}$ to HBM.
 - 3: Load $\mathbf{P}$ and $\mathbf{V}$ by blocks from HBM, compute $\mathbf{O} = \mathbf{P}\mathbf{V}$, write $\mathbf{O}$ to HBM.
 - 4: Return $\mathbf{O}$.
-

Core Q: How to tile softmax?

How to Implement Softmax

Algorithm 1 Naive softmax

```
1:  $d_0 \leftarrow 0$ 
2: for  $j \leftarrow 1, V$  do
3:    $d_j \leftarrow d_{j-1} + e^{x_j}$ 
4: end for
5: for  $i \leftarrow 1, V$  do
6:    $y_i \leftarrow \frac{e^{x_i}}{d_V}$ 
7: end for
```

Problem

- Can easily go overflow because of sum (e^x)

Safe Softmax

$$y_i = \frac{e^{x_i - \max_{k=1}^V x_k}}{\sum_{j=1}^V e^{x_j - \max_{k=1}^V x_k}}$$

Algorithm 2 Safe softmax

```
1:  $m_0 \leftarrow -\infty$ 
2: for  $k \leftarrow 1, V$  do
3:    $m_k \leftarrow \max(m_{k-1}, x_k)$ 
4: end for
5:  $d_0 \leftarrow 0$ 
6: for  $j \leftarrow 1, V$  do
7:    $d_j \leftarrow d_{j-1} + e^{x_j - m_V}$ 
8: end for
9: for  $i \leftarrow 1, V$  do
10:   $y_i \leftarrow \frac{e^{x_i - m_V}}{d_V}$ 
11: end for
```

Can we fuse?

Create alternative
sequence

$$d'_i := \sum_{j=1}^i e^{x_j - m_i}$$

With:
 $d'_V = d_V$

Algorithm 2 Safe softmax

```
1:  $m_0 \leftarrow -\infty$ 
2: for  $k \leftarrow 1, V$  do
3:    $m_k \leftarrow \max(m_{k-1}, x_k)$ 
4: end for
5:  $d_0 \leftarrow 0$ 
6: for  $j \leftarrow 1, V$  do
7:    $d_j \leftarrow d_{j-1} + e^{x_j - m_V}$ 
8: end for
9: for  $i \leftarrow 1, V$  do
10:   $y_i \leftarrow \frac{e^{x_i - m_V}}{d_V}$ 
11: end for
```

Further Note:

Create alternative
sequence

$$d'_i := \sum_{j=1}^i e^{x_j - m_i}$$

With:
 $d'_V = d_V$

The sequence exhibits
recurrence

$$\begin{aligned} d'_i &= \sum_{j=1}^i e^{x_j - m_i} \\ &= \left(\sum_{j=1}^{i-1} e^{x_j - m_i} \right) + e^{x_i - m_i} \\ &= \left(\sum_{j=1}^{i-1} e^{x_j - m_{i-1}} \right) e^{m_{i-1} - m_i} + e^{x_i - m_i} \\ &= d'_{i-1} e^{m_{i-1} - m_i} + e^{x_i - m_i} \end{aligned}$$

Online, Safe Softmax

Algorithm 3 Safe softmax with online normalizer calculation

```
1:  $m_0 \leftarrow -\infty$ 
2:  $d_0 \leftarrow 0$ 
3: for  $j \leftarrow 1, V$  do
4:    $m_j \leftarrow \max(m_{j-1}, x_j)$ 
5:    $d_j \leftarrow d_{j-1} \times e^{m_{j-1}-m_j} + e^{x_j-m_j}$ 
6: end for
7: for  $i \leftarrow 1, V$  do
8:    $y_i \leftarrow \frac{e^{x_i-m_V}}{d_V}$ 
9: end for
```

Q : can we further fuse these two loops?

Self attention v2: with Online Safe Softmax

NOTATIONS

$Q[k, :]$: the k -th row vector of Q matrix.

$K^T[:, i]$: the i -th column vector of K^T matrix.

$O[k, :]$: the k -th row of output O matrix.

$V[i, :]$: the i -th row of V matrix.

$\{\mathbf{o}_i\}$: $\sum_{j=1}^i a_j V[j, :]$, a row vector storing partial aggregation result $A[k, :i] \times V[:, i, :]$

BODY

for $i \leftarrow 1, N$ **do**

$$x_i \leftarrow Q[k, :] K^T[:, i]$$

$$m_i \leftarrow \max(m_{i-1}, x_i)$$

$$d'_i \leftarrow d'_{i-1} e^{m_{i-1} - m_i} + e^{x_i - m_i}$$

end

for $i \leftarrow 1, N$ **do**

$$a_i \leftarrow \frac{e^{x_i - m_N}}{d'_N}$$

$$\mathbf{o}_i \leftarrow \mathbf{o}_{i-1} + a_i V[i, :]$$

end

$$O[k, :] \leftarrow \mathbf{o}_N$$

Q1: can we further fuse these two loops?

Q2: How to get x_i ?

Self attention

NOTATIONS

$Q[k,:]$: the k -th row vector of Q matrix.

$K^T[:,i]$: the i -th column vector of K^T matrix.

$O[k,:]$: the k -th row of output O matrix.

$V[i,:]$: the i -th row of V matrix.

$\{\mathbf{o}_i\}$: $\sum_{j=1}^i a_j V[j,:]$, a row vector storing partial aggregation result $A[k,:i] \times V[i,:]$

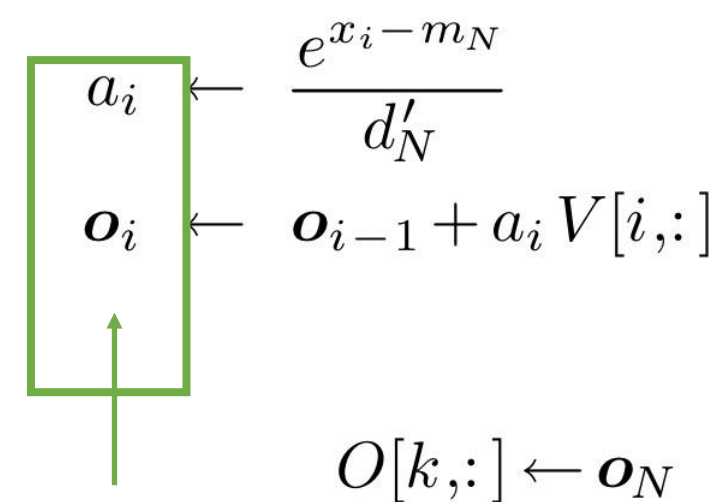
BODY

for $i \leftarrow 1, N$ **do**

$$\begin{aligned} x_i &\leftarrow Q[k,:] K^T[:,i] \\ m_i &\leftarrow \max(m_{i-1}, x_i) \\ d'_i &\leftarrow d'_{i-1} e^{m_{i-1}-m_i} + e^{x_i-m_i} \end{aligned}$$

end

for $i \leftarrow 1, N$ **do**



$$\begin{aligned} a_i &\leftarrow \frac{e^{x_i-m_N}}{d'_N} \\ \mathbf{o}_i &\leftarrow \mathbf{o}_{i-1} + a_i V[i,:] \end{aligned}$$

$O[k,:] \leftarrow \mathbf{o}_N$

end

$$\mathbf{o}_i := \sum_{j=1}^i \left(\frac{e^{x_j-m_N}}{d'_N} V[j,:] \right)$$

Create alternative sequence with $O_N = \mathbf{o}'_N$

$$\mathbf{o}'_i := \left(\sum_{j=1}^i \frac{e^{x_j-m_i}}{d'_i} V[j,:] \right)$$

in self attention, we just want o, not a

Derive the Recurrence

$$\begin{aligned} \mathbf{o}'_i &= \sum_{j=1}^i \frac{e^{x_j - m_i}}{d'_i} V[j, :] \\ &= \left(\sum_{j=1}^{i-1} \frac{e^{x_j - m_i}}{d'_i} V[j, :] \right) + \frac{e^{x_i - m_i}}{d'_i} V[i, :] \\ &= \left(\sum_{j=1}^{i-1} \frac{e^{x_j - m_{i-1}}}{d'_{i-1}} \frac{e^{x_j - m_i}}{e^{x_j - m_{i-1}}} \frac{d'_{i-1}}{d'_i} V[j, :] \right) + \frac{e^{x_i - m_i}}{d'_i} V[i, :] \\ &= \left(\sum_{j=1}^{i-1} \frac{e^{x_j - m_{i-1}}}{d'_{i-1}} V[j, :] \right) \frac{d'_{i-1}}{d'_i} e^{m_{i-1} - m_i} + \frac{e^{x_i - m_i}}{d'_i} V[i, :] \\ &= \mathbf{o}'_{i-1} \frac{d'_{i-1} e^{m_{i-1} - m_i}}{d'_i} + \frac{e^{x_i - m_i}}{d'_i} V[i, :] \end{aligned}$$

Insight: $\mathbf{o}'_i$ only depends on:
 $\mathbf{o}'_{i-1}, d'_{i-1}, d'_i, m_i, m_{i-1}$

Finally: Flash Attention

1. we only read x_1 (Q, K) once
2. We never materialize the full S
3. we never materialize the full a (softmax(S))

for $i \leftarrow 1, N$ **do**

$$x_i \leftarrow Q[k, :] K^T[:, i]$$

$$m_i \leftarrow \max(m_{i-1}, x_i)$$

$$d'_i \leftarrow d'_{i-1} e^{m_{i-1} - m_i} + e^{x_i - m_i}$$

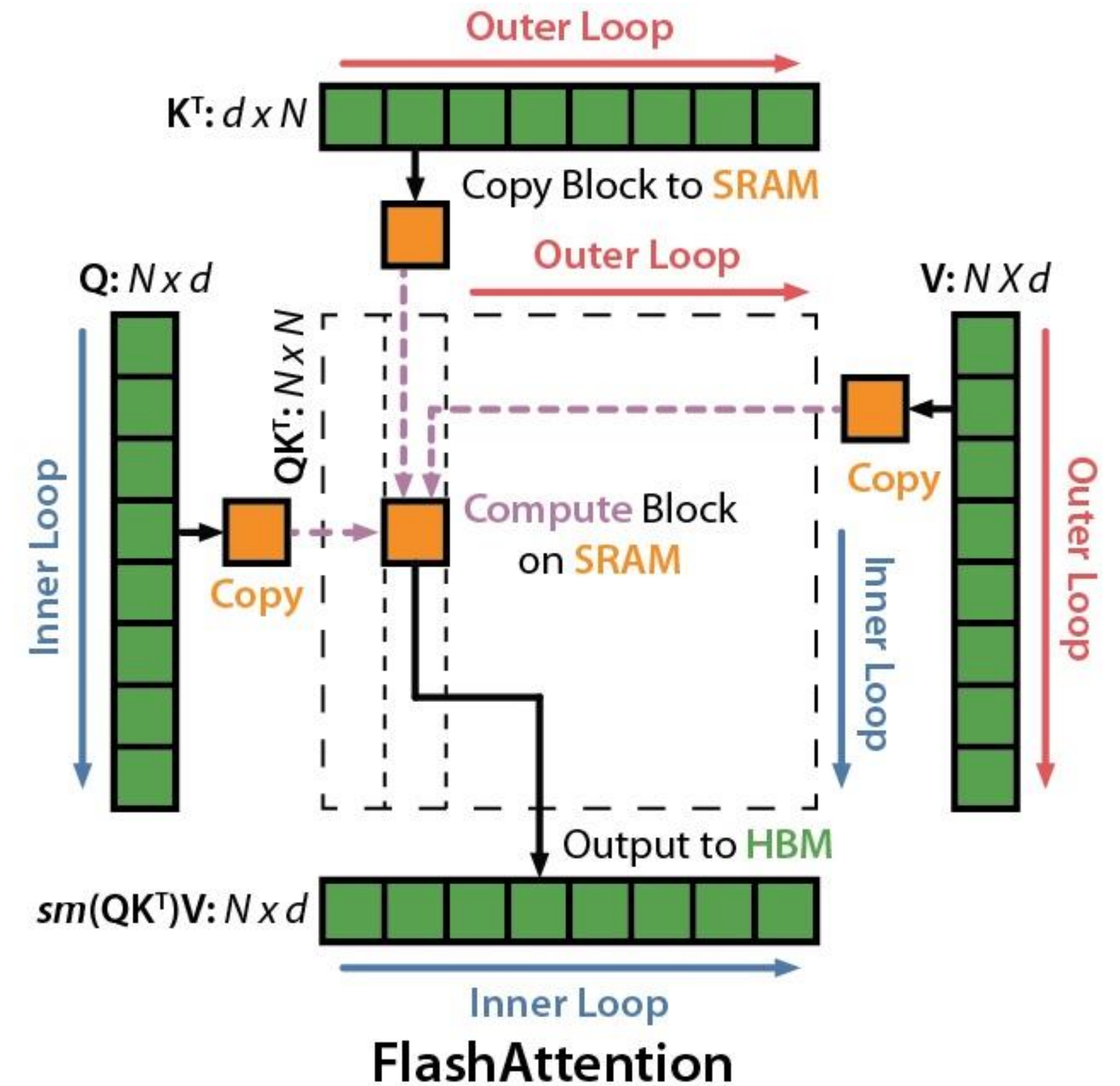
$$\mathbf{o}'_i \leftarrow \mathbf{o}'_{i-1} \frac{d'_{i-1} e^{m_{i-1} - m_i}}{d'_i} + \frac{e^{x_i - m_i}}{d'_i} V[i, :]$$

end

$$O[k, :] \leftarrow \mathbf{o}'_N$$

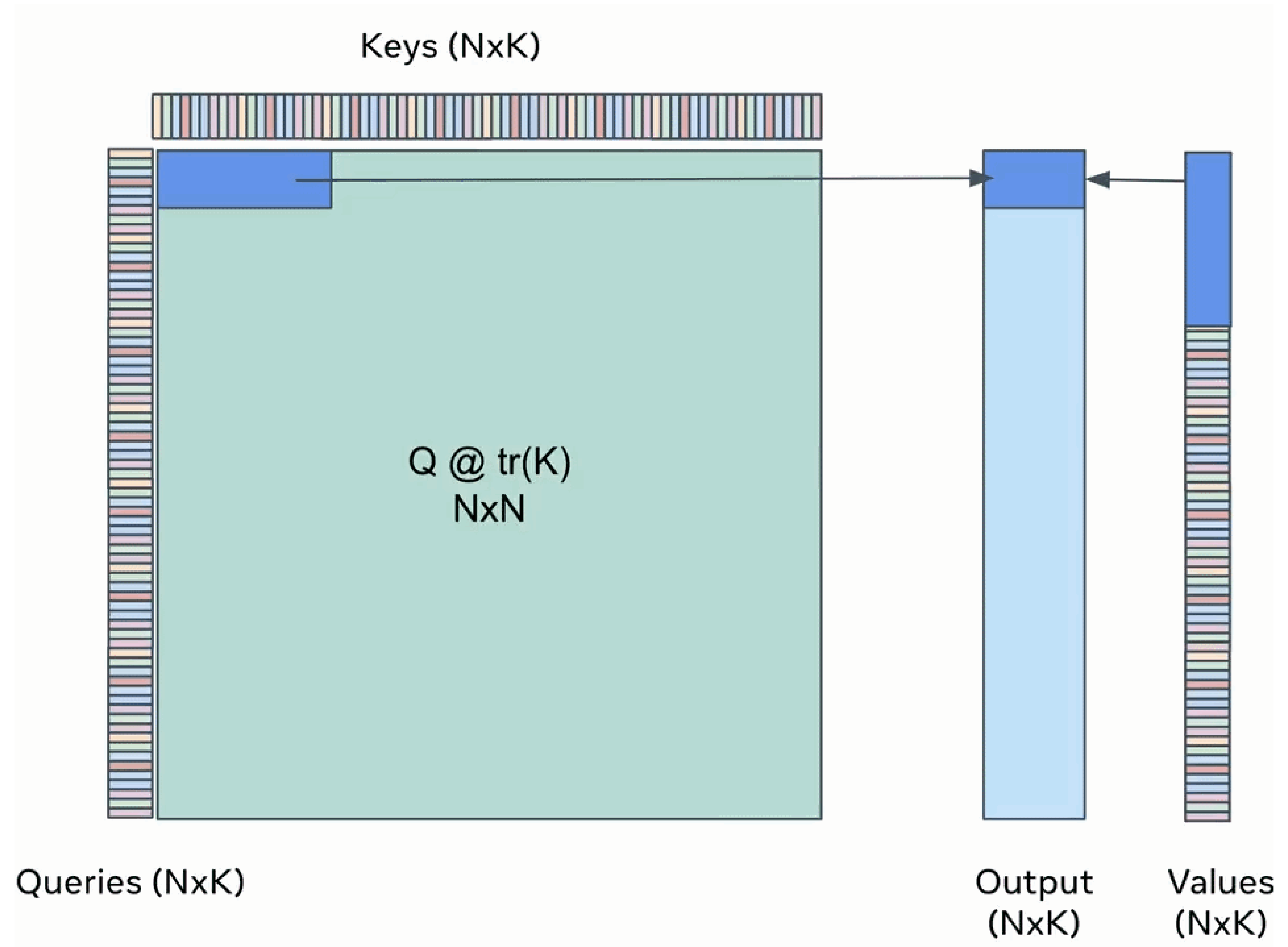
Tiling: Decompose Large Softmax into smaller ones by Scaling

1. Load inputs by blocks from global to shared memory
2. On chip, compute attention output wrt the block
3. Update output in device memory by scaling



+ Tiling

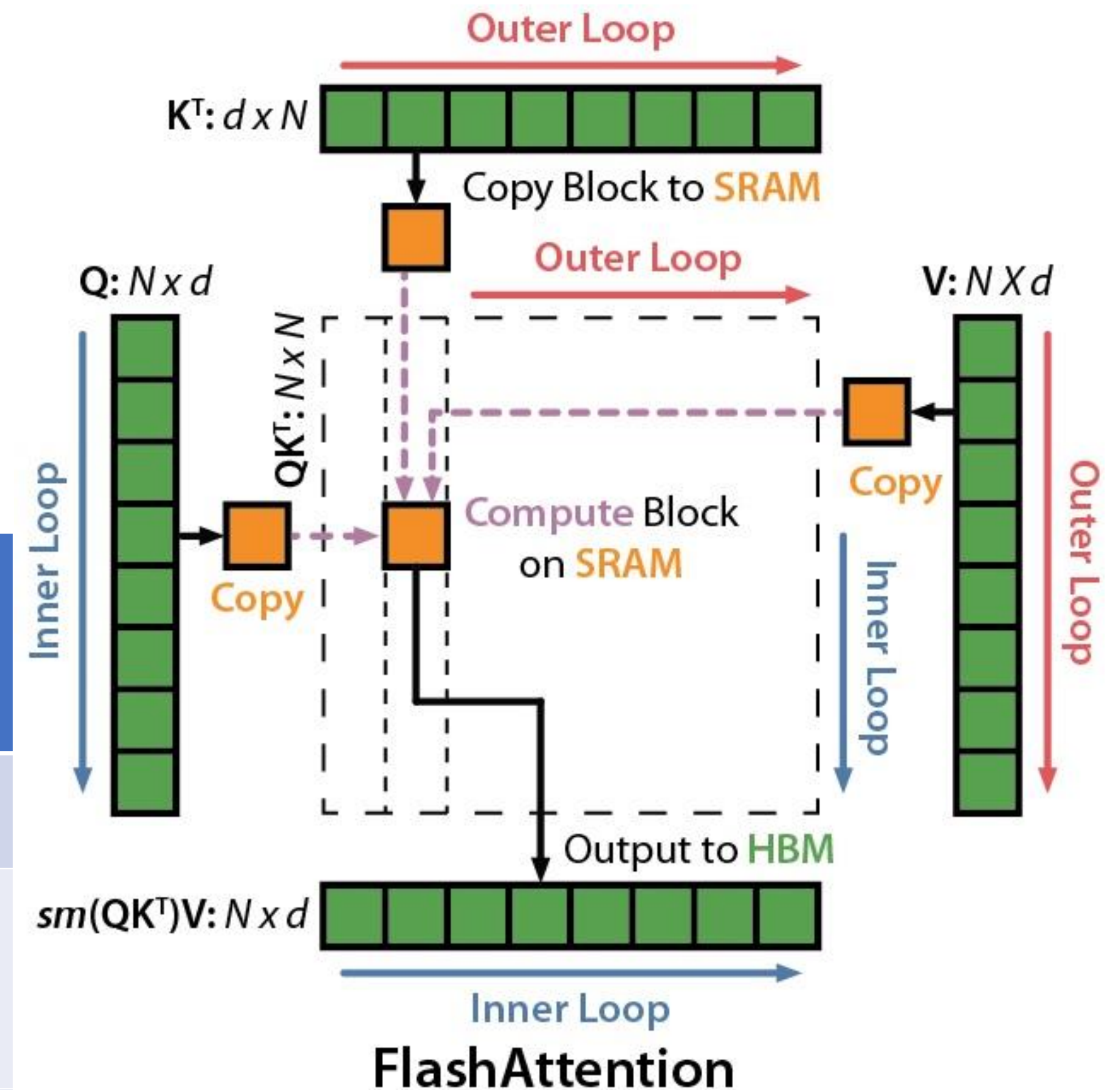
1. Load inputs by blocks from global to shared memory
2. On chip, compute attention output wrt the block
3. Update output in device memory by scaling



Recomputation: Backward Pass

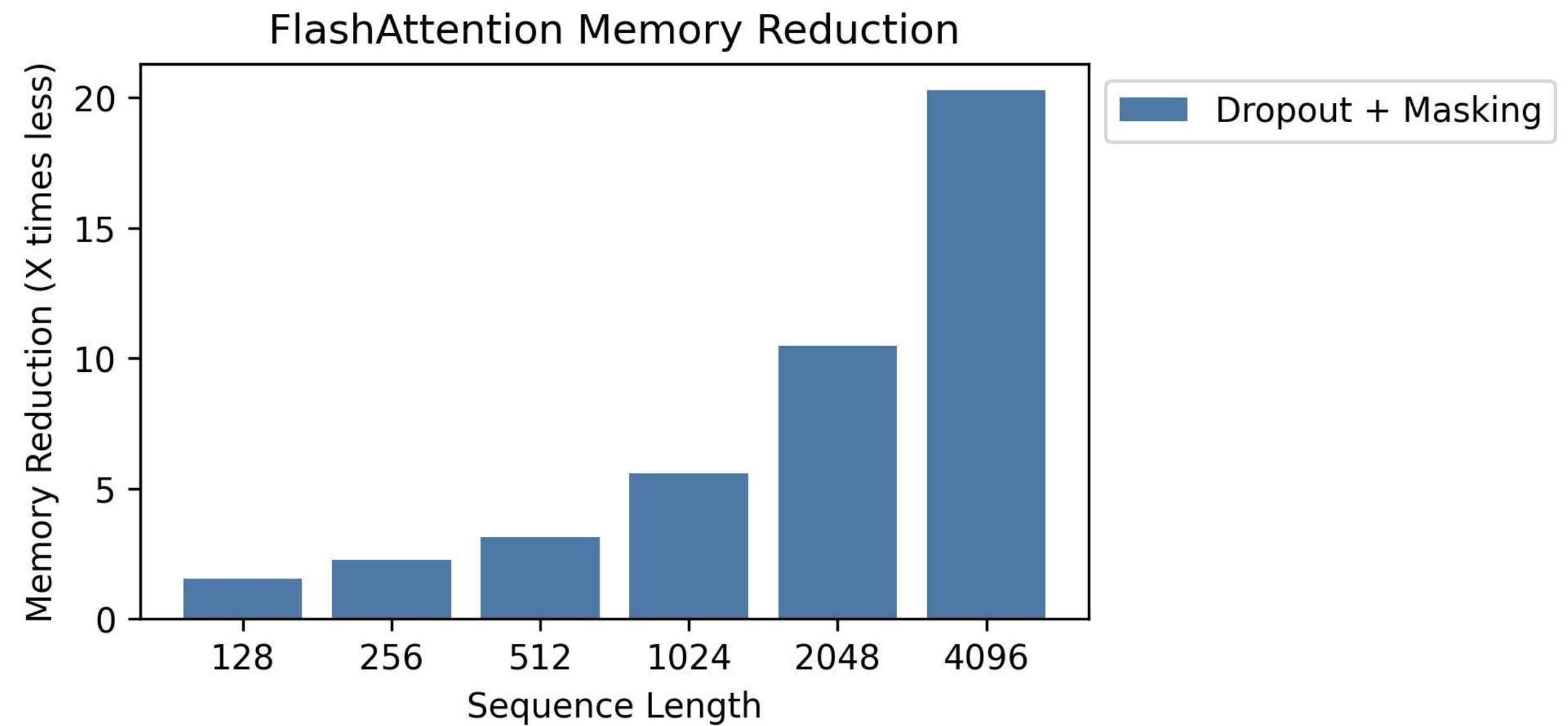
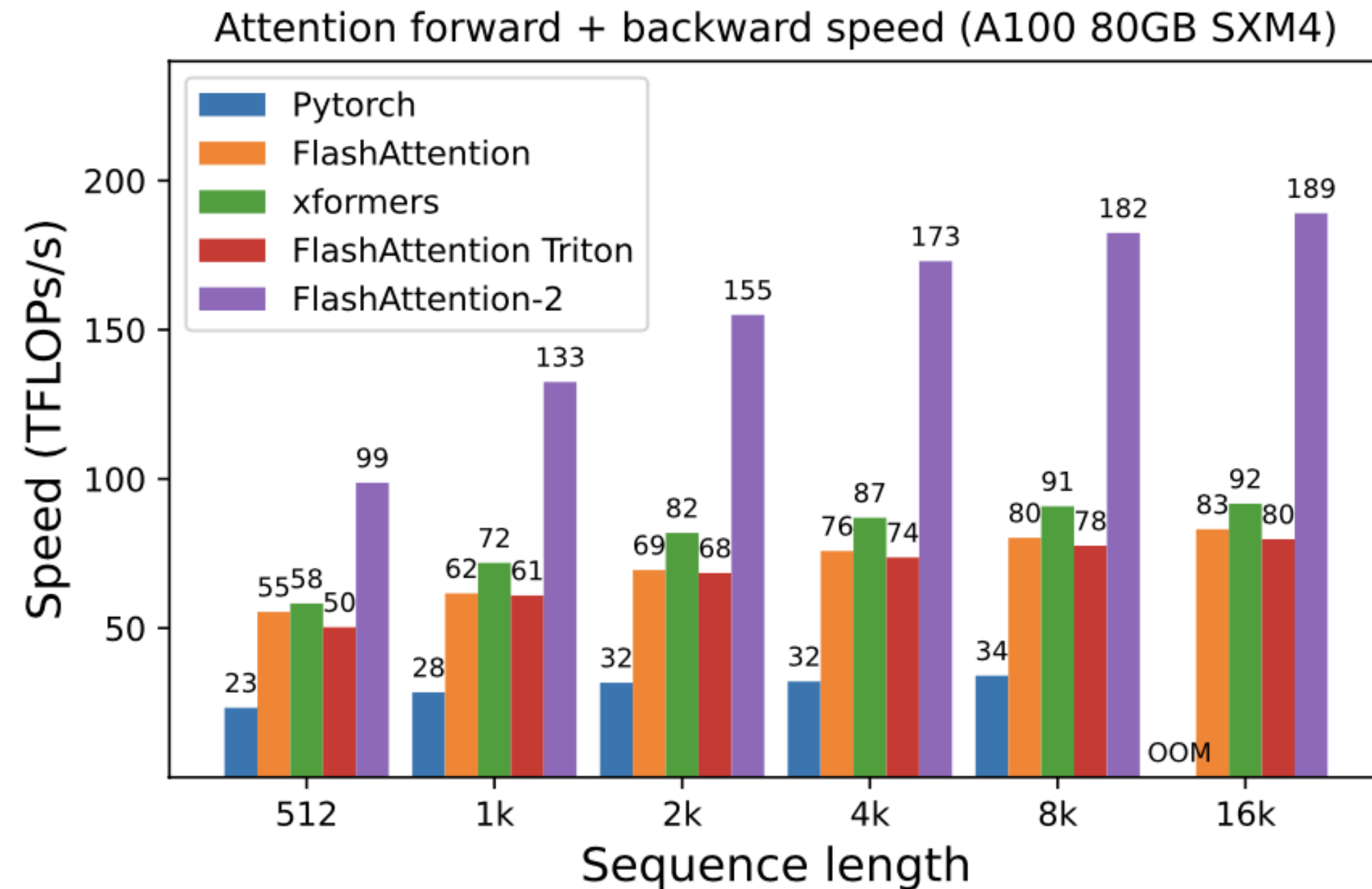
By storing softmax normalization factors from forward (size N), recompute attention in the backward from inputs in

Attention	Standard	FlashAttention
GFLOPs	66.6	75.2
Global mem access	40.3 GB	4.4 GB
Runtime	41.7 ms	7.3 ms



Speed up backward pass with with increased FLOPs!

FlashAttention: 2-4x speedup, 10-20x memory reduction



High-level GISTs of FA

- It completely avoid materializing the large S of size bs^2n
- It will make the compute $4bs^2h$ even worse
- It will greatly reduce memory movement between memory hierarchy

Why FA is even greater win than thought -- Cascaded effect:

- Because it saves the memory of bs^2n , it enables two possibilities
 - Can train with a large b (in fact, pre-FA, most train with $b=1$)
 - -> high AI
 - Can turn off gradient checkpointing
 - -> No need to pay extra forward (25% flops)

FA2 and FA3

- More kernel-level optimizations over FA
 - More fusion
 - Improved memory access patterns
- Discussion: why FA took off?

large b



Large Language Models

- Transformers, Attentions
- Scaling Law
 - MoE
- Connecting the dots: Training Optimizations
 - **Flash attention** ← **come back to this later next week**
- Serving and inference optimization
 - Continuous batching and Paged attention
 - Speculative decoding (Guest Lecture)
- Connecting the dots: Deepseek-v3
- Hot topics